
RAPPORT DE TRAVAUX PRATIQUES

BASE DE DONNÉES RELATIONNELLES NIVEAU 2

Master Management des systèmes d'information

Présentée par :

BENNAÇAR Salma 12517414

BEN ZIKI Nahid 12504024

GOBALASAMY Ashani 12200679

Sous la direction de :

BERNARD Camille

TABLE DE MATIÈRES

I. INTRODUCTION.....	2
II. DICTIONNAIRES DE DONNÉES.....	3
III. MODÈLE ENTITÉ-ASSOCIATION, MODÈLE LOGIQUE DE DONNÉES ET DIAGRAMME EER.....	6
IV. DOCUMENTATION DÉTAILLÉE.....	8
1. CRÉATION DE LA BASE DE DONNÉES ET DES TABLES.....	8
- Les fournisseurs.....	8
- Les équipements.....	8
- La facturation côté fournisseurs.....	9
2. INSERTION DES DONNÉES.....	10
3. JUSTIFICATION DES CHOIX DE FONCTIONNALITÉS AVANCÉES.....	10
a) Pourquoi ces requêtes SELECT ?.....	10
b) Pourquoi ces vues ?.....	10
c) Pourquoi ces triggers ?.....	10
d) Pourquoi ces procédures ?.....	11
e) Pourquoi ces fonctions ?.....	11
V. PRINCIPAUX RÉSULTATS.....	12
Une base de données structurée et cohérente.....	12
Un jeu de données réaliste.....	12
Des fonctionnalités avancées opérationnelles.....	12
VI. DIFFICULTÉS RENCONTRÉES.....	13
VII. IMPLICATION DE CHAQUE MEMBRE.....	14
Ben Ziki Nahid.....	14
Gobalasamy Ashani.....	14
Bennacar Salma.....	14
VIII. CONCLUSION.....	15

I. INTRODUCTION

Ce projet a pour objectif de concevoir et de mettre en place le système d'information de la station SnowPeak. L'idée était de créer une base de données capable de centraliser toutes les activités de la station : de la location de matériel à l'organisation des cours de ski, en passant par le suivi des employés et de la facturation.

Pour mener à bien ce TP, nous avons structuré le travail en trois grandes étapes :

- La modélisation et la structure : à travers la création de l'architecture de la base (les tables) en s'assurant que les relations entre les clients, les stations et les équipements soient cohérentes et solides.
- La mise en situation : par l'intégration d'un jeu de données complet pour simuler l'activité réelle de la station. Cette étape était essentielle pour vérifier que le modèle de données fonctionnait bien avec des cas concrets.
- L'automatisation et l'analyse : dans le but de rendre la base de données vraiment utile pour les utilisateurs grâce à des triggers et procédures qui gèrent les stocks et les paiements tout seuls, tout en simplifiant l'affichage des données importantes via des vues et des rapports de sélection.

L'objectif final de cette démarche est de proposer une solution qui ne se contente pas de stocker des informations, mais qui aide réellement la station dans sa gestion quotidienne.

II. DICTIONNAIRES DE DONNÉES

Afin de garantir la cohérence du système, ce dictionnaire définit précisément chaque information stockée, facilitant ainsi la maintenance et l'évolution de la base de données

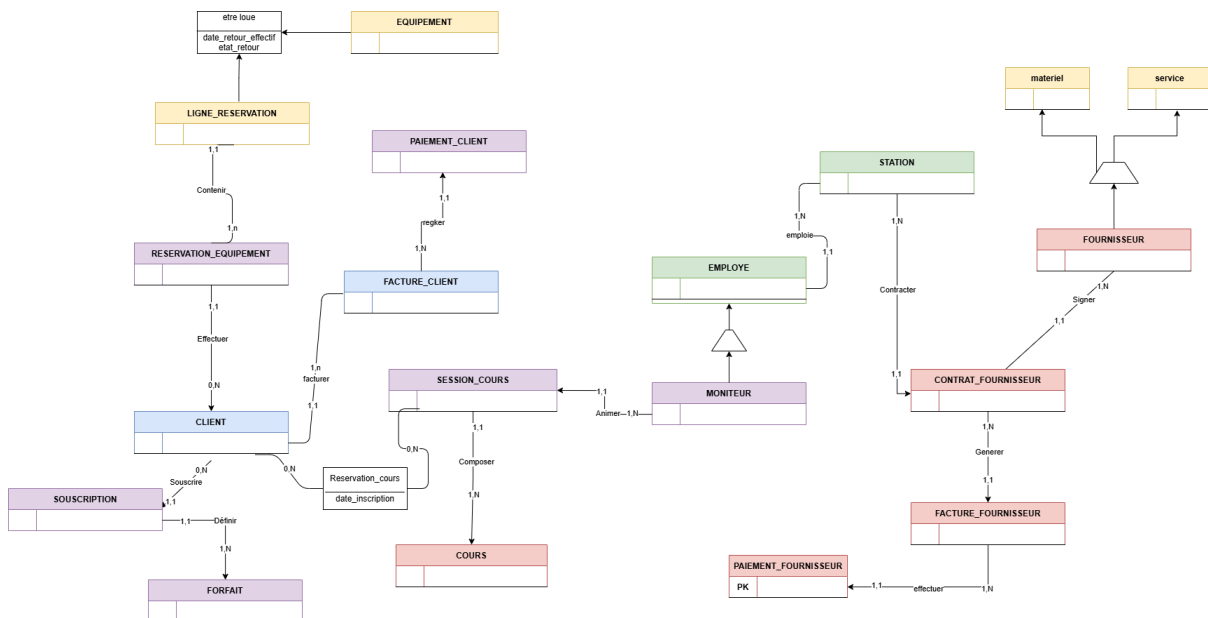
Table	Nom du champ	Type	Description
station	id_station	INT	Identifiant unique de la station (Clé primaire)
	nom_station	VARCHAR(100)	Nom de la station de ski
	altitude_m	INT	Altitude de la station en mètres
	ville	VARCHAR(100)	Ville de localisation
fournisseur	id_fournisseur	INT	Identifiant unique du fournisseur (Clé primaire)
	nom_societe	VARCHAR(100)	Nom de l'entreprise ou raison sociale
	contact_nom	VARCHAR(100)	Nom de l'interlocuteur chez le fournisseur
	tel_pro	VARCHAR(20)	Numéro de téléphone professionnel
fourn_materiel	id_fournisseur	INT	Identifiant (Spécialisation fournisseur matériel)
	marques_proposees	VARCHAR(100)	Liste des marques distribuées
fourn_services	id_fournisseur	INT	Identifiant (Spécialisation fournisseur services)
	type_prestation	ENUM	Type de service : ASSURANCE_NEIGE, ASSISTANCE_PISTE, PACK_COMPLET
contrat_fournisseur	id_contrat	INT	Identifiant unique du contrat (Clé primaire)
	date_debut	DATE	Date de début de validité du contrat
	date_fin	DATE	Date de fin (NULL = reconduit tacitement)
employe	id_employe	INT	Identifiant unique de l'employé (Clé primaire)

	nom	VARCHAR(50)	Nom de famille
	prenom	VARCHAR(50)	Prénom
moniteur	id_employe	INT	Identifiant (Spécialisation employé moniteur)
	num_diplome	VARCHAR(20)	Numéro de diplôme unique du moniteur
client	id_client	INT	Identifiant unique du client (Clé primaire)
	nom	VARCHAR(50)	Nom de famille
	prenom	VARCHAR(50)	Prénom
	date_naissance	DATE	Date de naissance (calcul de la tranche tarifaire)
	email	VARCHAR(100)	Adresse mail unique du client
	telephone	VARCHAR(20)	Numéro de téléphone
	adresse	VARCHAR(100)	Adresse postale
forfait	id_forfait	INT	Identifiant unique du type de forfait (Clé primaire)
	libelle	VARCHAR(50)	Nom du forfait (ex. Pass Saison, Journée)
	prix_unitaire	DECIMAL(10,2)	Tarif de base du forfait (doit être > 0)
souscription_forfait	id_souscription	INT	Identifiant unique de la souscription (Clé primaire)
	date_achat	DATE	Date de la transaction
	date_debut	DATE	Date de début de validité du forfait
	date_fin	DATE	Date de fin de validité du forfait
cours	id_cours	INT	Identifiant unique du type de cours (Clé primaire)
	niveau	VARCHAR(20)	Niveau requis (ex. Débutant, Intermédiaire)
session_cours	id_session	INT	Identifiant d'un créneau de cours spécifique (Clé primaire)

	nb_places_restantes	INT UNSIGNED	Nombre de places encore disponibles
reservation_cours	id_client	INT	Identifiant du client inscrit (Clé primaire composite)
	id_session	INT	Identifiant de la session réservée (Clé primaire composite)
	date_inscription	DATETIME	Date et heure de l'inscription
equipement	id_equipement	INT	Identifiant unique de l'article (Clé primaire)
	type_mat	VARCHAR(50)	Type de matériel (ex. Ski Rossignol, Snowboard Burton)
	etat	ENUM	État du matériel : NEUF, BON, USURE, HORS_SERVICE
	disponibilite	BOOLEAN	Disponibilité en stock (TRUE = disponible, FALSE = loué)

III. MODÈLE ENTITÉ-ASSOCIATION, MODÈLE LOGIQUE DE DONNÉES ET DIAGRAMME EER

MODÈLE ENTITÉ ASSOCIATION



MODÈLE LOGIQUE DE DONNÉES

Station (id_station, nom_station, altitude_m, ville)

Employe (id_employe, nom, prenom, #id_station)

Moniteur (#id_employe, num_diplome, specialite)

Client (id_client, nom, prenom, email, telephone, adresse)

Forfait (id_forfait, libelle, prix_unitaire, type_acces)

Souscription_Forfait (id_souscription, date_achat, date_debut, date_fin, #id_client, #id_forfait)

Cours (id_cours, libelle, niveau_requis, description)

Session_Cours (id_session, date_heure, max_participants, #id_cours, #id_employe, nb_places_restantes)

Reservation_Cours (#id_client, #id_session, date_inscription)

Equipement (id_equipement, type_mat, modele, taille, etat_actuel, #id_station)

Reservation_Equipement (id_resa_mat, date_resa, #id_client)

Ligne_Reservation (id_ligne, #id_resa_mat, #id_equipement, date_retour_prevue, date_retour_reel, etat_retour, prix_applique)

Facture_Client (id_facture_c, date_emission, total_ht, taux_tva, #id_client)

Paielement_Client (id_paiement_c, montant, date_paiement, mode_paiement, #id_facture_c)

Fournisseur (id_fournisseur, nom_societe, contact_nom, tel_pro)

Fourn_Materiel (#id_fournisseur, marques_proposees)

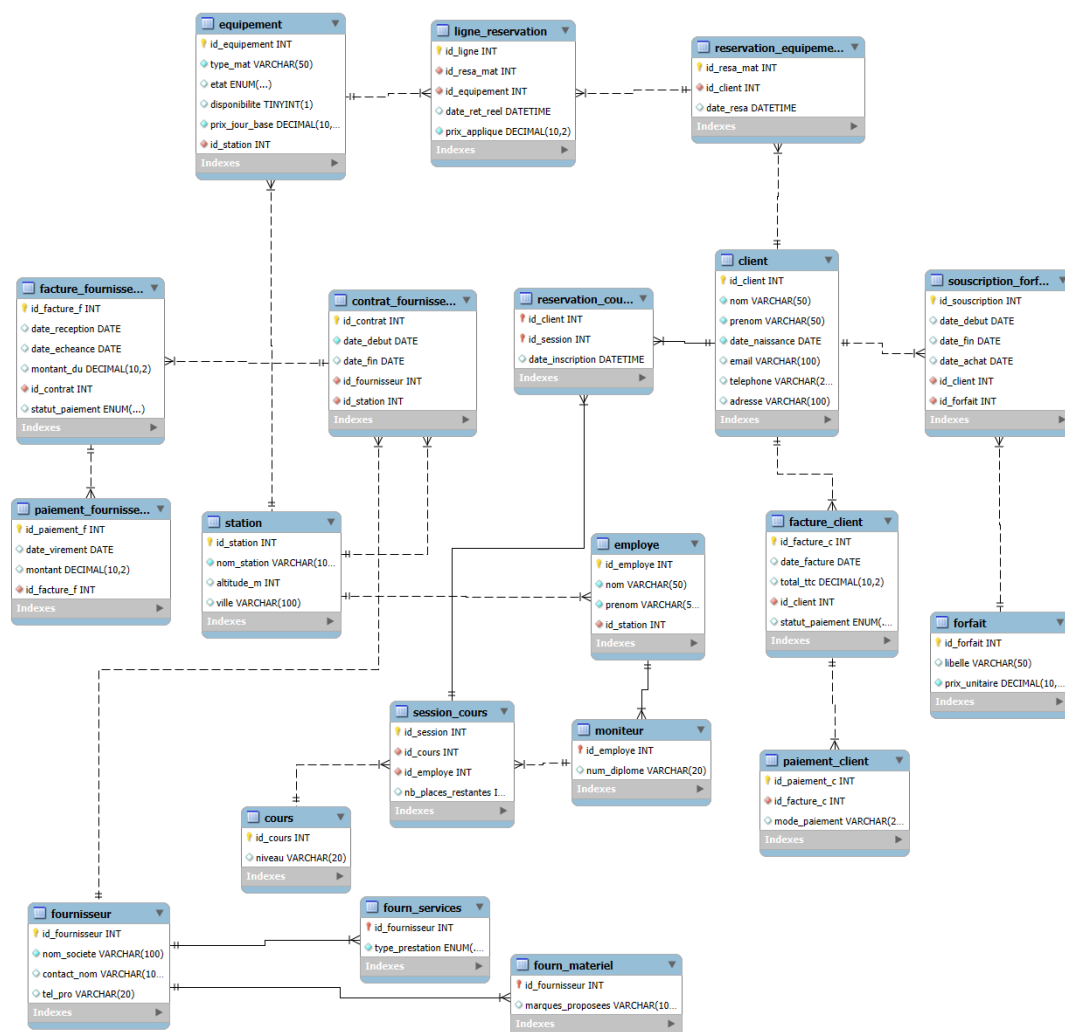
Fourn_Services (#id_fournisseur, type_prestation)

Contrat_Fournisseur (id_contrat, date_debut, date_fin, #id_fournisseur, #id_station)

Facture_Fournisseur (id_facture_f, date_reception, montant_du, #id_contrat)

Paielement_Fournisseur (id_paiement_f, date_virement, montant, #id_facture_f)

DIAGRAMME EER



IV. DOCUMENTATION DÉTAILLÉE

La base de données SnowPeak permet de gérer tous les aspects d'une société qui exploite plusieurs stations de ski. On y retrouve la gestion des stations, des équipements à louer, des clients, des réservations, des cours de ski, des fournisseurs et de la facturation.

1. CRÉATION DE LA BASE DE DONNÉES ET DES TABLES

Elle contient 18 tables organisées autour de trois hiérarchies d'héritage :

- **Les stations**

La table station est le point central du modèle. Presque tout y est rattaché : les équipements, les employés, les cours, les contrats fournisseurs. Elle contient juste le nom, la ville et l'altitude de chaque station.

- **Les fournisseurs**

On a deux types de fournisseurs chez SnowPeak : ceux qui s'occupent du matériel et de la maintenance, et ceux qui fournissent les services d'assurance et d'assistance sur les pistes. Ces deux types n'ont pas du tout les mêmes informations, donc on a créé deux sous-tables (fourn_materiel et fourn_services) qui héritent des informations de base de la table fournisseur. Si on avait tout mis dans une seule table, on aurait eu plein de colonnes vides selon le type de fournisseur, ce qui n'est pas propre. L'héritage règle ce problème.

Le lien entre un fournisseur et une station passe par la table contrat_fournisseur, qui stocke la période de validité du contrat. Un fournisseur peut travailler avec plusieurs stations, et une station peut avoir plusieurs fournisseurs.

- **Les employés**

Même logique que pour les fournisseurs. Un moniteur dispose d'informations supplémentaires qu'un technicien ou un administratif n'a pas, notamment son numéro de diplôme. On a donc créé une sous-table moniteur qui hérite de employe.

On n'a pas créé de sous-table pour les techniciens et administratifs car ils n'ont pas d'attributs ou de relations spécifiques dans notre modèle, leur présence dans employe suffit.

- **Les clients**

La table client stocke les informations personnelles des clients. On a fait le choix de stocker la date de naissance plutôt qu'une tranche d'âge, car l'âge change chaque année. Stocker directement une tranche d'âge violerait la 3NF puisque cette information se calcule à partir de la date de naissance. On la calcule donc dynamiquement quand on en a besoin, notamment pour appliquer le bon tarif.

- **Les équipements**

Tous les équipements (skis, snowboards, luges) sont dans une seule table equipement. On aurait pu créer une table par type, mais leurs attributs sont presque identiques (état, disponibilité, prix, station), donc une seule table avec un champ type_mat est largement suffisant.

Le champ disponibilite est automatiquement mis à jour par des triggers : il passe à FALSE quand un équipement est loué, et revient à TRUE quand il est rendu.

Le prix_jour_base est le tarif de référence. Quand une location est créée, ce prix est copié dans prix_applique dans la table ligne_reservation. Ainsi, si on change le tarif d'un équipement plus tard, ça ne change pas les locations déjà enregistrées. Nous avons pensé que c'était important d'un point de vue comptable.

- **Les réservation d'équipements**

La réservation d'équipements est découpée en deux tables. `reservation_equipement` est l'en-tête : elle dit quel client réserve et quand. `ligne_reservation` contient le détail : quel équipement précis a été loué, à quel prix, et quand il a été rendu.

Ce découpage permet de gérer le cas réaliste où un client loue plusieurs équipements en même temps et les rend à des dates différentes comme par exemple s' il rend les skis le jour 3 mais garde les bâtons jusqu'au jour 5.

- Les cours et réservations des cours

La table `cours` est le catalogue : elle décrit les types de cours proposés. La table `session_cours` représente un créneau concret de ce cours, avec un moniteur et un nombre de places précis.

Nous avons de séparer de cette façon car un même cours peut avoir plusieurs créneaux par semaine avec des moniteurs différents. Sans ça, on serait obligé de dupliquer les informations du cours pour chaque créneau.

La table `reservation_cours` enregistre les inscriptions. Sa clé primaire est composite (`id_client`, `id_session`), ce qui interdit à un même client de s'inscrire deux fois à la même session.

- Les forfaits

La table `forfait` décrit les forfaits d'accès aux pistes disponibles. La table `souscription_forfait` enregistre l'achat d'un forfait par un client avec ses dates de validité.

- La facturation côté clients

La table `facture_client` regroupe les factures émises aux clients. Le statut de paiement (`NON_PAYE`, `PARTIEL`, `PAYE`) sera mis à jour automatiquement par trigger quand un paiement est enregistré dans `paiement_client`.

On a séparé la facture et le paiement pour permettre les règlements partiels ou en plusieurs fois, ce qui arrive régulièrement en station notamment pour les groupes.

- La facturation côté fournisseurs

La table `facture_fournisseur` stocke les factures reçues des fournisseurs dans le cadre d'un contrat. On la rattache à `contrat_fournisseur` plutôt qu'à `fournisseur` directement, pour savoir précisément dans quel contexte et pour quelle station la prestation a été réalisée.

Le statut (`A_PAYER`, `EN_COURS`, `REGLE`) sera lui aussi mis à jour automatiquement par trigger dès qu'un paiement est enregistré dans `paiement_fournisseur`.

Le schéma respecte la troisième forme normale. Concrètement, ça veut dire que chaque information n'est stockée qu'à un seul endroit. Par exemple, on ne stocke pas la tranche d'âge d'un client car elle se déduit de sa date de naissance, la recalculer évite d'avoir une donnée qui peut devenir fausse avec le temps. De même, `prix_applique` dans `ligne_reservation` est une copie figée du tarif au moment de la location, et non une référence au tarif actuel de l'équipement, pour éviter qu'une modification de tarif change l'historique des locations.

Toutes les relations entre tables sont protégées par des clés étrangères. On a utilisé trois politiques selon les cas :

`ON UPDATE CASCADE` propage automatiquement les modifications d'identifiants dans les tables liées. `ON DELETE RESTRICT` empêche de supprimer une entité tant qu'elle est encore utilisée ailleurs par exemple on ne peut pas supprimer un client qui a encore des réservations. `ON DELETE CASCADE` est réservé aux sous-tables d'héritage : si on supprime un fournisseur de la table principale, sa ligne dans `fourn_materiel` ou `fourn_services` est supprimée automatiquement, ce qui est logique.

Des contraintes CHECK viennent compléter le tout : les prix doivent être positifs, les valeurs ENUM sont vérifiées à l'insertion, et les champs obligatoires sont déclarés NOT NULL.

2. INSERTION DES DONNÉES

Pour les insertions, nous avons décidé de ne pas nous contenter de 2 ou 3 exemples par table. Nous avons choisi d'insérer au moins 8 lignes pour chaque catégorie principale (8 clients, 8 forfaits, 8 réservations, etc.), ce qui nous fait dépasser les 100 lignes au total.'

Nous avons fait ce choix pour deux raisons :

1. Avoir des résultats qui parlent : En mettant beaucoup de données, nos requêtes SELECT deviennent plus intéressantes. Par exemple, pour le calcul du chiffre d'affaires ou l'état des stocks, c'est plus flagrant de voir si ça marche quand il y a beaucoup de lignes à additionner.
2. Pouvoir comparer : Avec 8 insertions minimum, on peut vraiment comparer les chiffres entre les trois stations, voir quel matériel est le plus loué et vérifier que nos filtres fonctionnent bien.

3. JUSTIFICATION DES CHOIX DE FONCTIONNALITÉS AVANCÉES

a) Pourquoi ces requêtes SELECT ?

Pour choisir nos SELECT, on s'est posé une question simple : si on travaillait dans une station de ski, quelles informations on aurait besoin au quotidien ?

On a identifié trois besoins principaux. D'abord le suivi du matériel pour savoir ce qui est disponible, ce qui est en location, ce qui n'est jamais loué. Ensuite le suivi financier dans le but de voir les factures impayées côté clients et côté fournisseurs. Et enfin la gestion des équipes afin de suivre la charge de travail des moniteurs et savoir qui anime les cours.

On a aussi voulu montrer qu'on maîtrisait les jointures multiples et les fonctions d'agrégation comme GROUP BY, COUNT, SUM ou GROUP_CONCAT, donc on a fait des requêtes un peu plus complexes comme le tableau de bord des stations.

b) Pourquoi ces vues ?

On a créé des vues pour les requêtes qu'on utiliserait le plus souvent. On a réfléchi aux différents utilisateurs de la base. Un agent de location consultera v_stock_equipement et v_locations_en_cours plusieurs fois par jour. Le service commercial utilisera v_fiche_client pour voir l'activité de chaque client et proposer des offres. La direction regardera v_fiche_station pour avoir un état des lieux rapide. La comptabilité utilisera v_factures_clients_impayees pour faire ses relances.

L'idée c'était donc de créer une vue par profil d'utilisateur, en se demandant ce dont chaque personne a besoin dans son travail.

c) Pourquoi ces triggers ?

On a choisi les triggers pour automatiser les mises à jour qui doivent se faire à chaque fois qu'une donnée change, sans qu'on ait besoin d'y penser.

Le cas le plus évident c'est la disponibilité du matériel. Quand un équipement est loué, il doit passer à disponible = FALSE automatiquement. Quand il est rendu, il repasse à TRUE. Même logique pour les places de cours : à chaque inscription le compteur diminue, à chaque annulation il augmente.

Pour les factures, on a voulu montrer deux niveaux. Le trigger fournisseur est plus complexe, il calcule le cumul des paiements avec des variables DECLARE et choisit entre EN_COURS et REGLE. Le trigger client est plus simple car la table paiement_client n'a pas de colonne montant dans notre schéma.

d) Pourquoi ces procédures ?

On a créé des procédures pour les opérations qui touchent plusieurs tables en même temps et qui ont besoin de vérifications avant d'agir.

sp_reserver_equipement et sp_restituer_equipement couvrent le cœur du métier : louer et récupérer du matériel. Sans ces procédures, on pourrait créer une réservation sur un équipement déjà loué ou hors service. La procédure vérifie tout ça avant d'insérer quoi que ce soit.

sp_inscrire_cours et sp_annuler_inscription_cours font la même chose pour les cours: elles centralisent toutes les vérifications en un seul endroit plutôt que de les disperser dans plusieurs requêtes.

sp_creeer_facture_client montre qu'une procédure peut aussi faire des calculs automatiques, ici le total des locations d'un client, pour éviter les erreurs de saisie.

e) Pourquoi ces fonctions ?

On a voulu aller un peu plus loin que ce qui était demandé en explorant ce qu'on pouvait faire avec les fonctions dans le contexte de SnowPeak.

fn_age_client est la plus utile : plutôt que de recalculer l'âge du client dans chaque requête, on le centralise dans une fonction réutilisable. C'est important pour la tarification car les prix varient selon l'âge.

fn_nb_equip_disponibles et fn_ca_client sont des fonctions qu'on imaginerait appeler depuis une application pour afficher le stock disponible en temps réel ou le chiffre d'affaire d'un client pour le programme de fidélité.

fn_montant_du_fournisseur répond à un besoin comptable simple : savoir en un appel combien on doit encore à chaque fournisseur.

fn_station_la_plus_active est la plus analytique. Elle agrège des données sur toute la base pour sortir un indicateur de performance.

V. PRINCIPAUX RÉSULTATS

Ce projet nous a permis de concevoir et de déployer une base de données relationnelle complète pour la société SnowPeak, société fictive de gestion de stations de ski. Voici les principaux résultats obtenus.

Une base de données structurée et cohérente

La base de données SnowPeak comprend 18 tables organisées autour de trois grands axes : la gestion des stations et des équipements, la gestion des clients et des réservations, et la gestion des fournisseurs et de la facturation.

Trois hiérarchies d'héritage ont été implémentées pour refléter fidèlement le modèle métier :

- Fournisseur -> Fourn_Materiel / Fourn_Services
- Employé -> Moniteur
- Réservation d'équipement -> Ligne_Reservation (modele en-tete / lignes)

Toutes les relations entre tables sont sécurisées par des clés étrangères avec des politiques ON UPDATE CASCADE et ON DELETE RESTRICT ou CASCADE selon les cas. Des contraintes CHECK garantissent l'intégrité des données métier, notamment sur les prix et les valeurs ENUM.

Un jeu de données réaliste

Le script d'insertion contient des données de test cohérentes et diversifiées pour simuler l'activité réelle d'une station de ski :

- 3 stations de ski dans différentes régions françaises
- 4 fournisseurs dont 2 spécialisés matériel et 2 en services
- 8 contrats fournisseurs liant stations et prestataires
- 8 employés dont 6 moniteurs certifiés
- 8 clients de nationalités variées
- 8 forfaits d'accès aux pistes à différents tarifs
- 8 cours et 8 sessions planifiées avec des moniteurs assignés
- 12 équipements répartis dans les stations avec état et disponibilité
- 8 réservations d'équipements et 8 lignes de location
- 8 factures clients et 8 factures fournisseurs avec leurs paiements associés

Des fonctionnalités avancées opérationnelles

Le script avance implémente des fonctionnalités permettant d'automatiser la gestion quotidienne de la station, notamment la mise à jour automatique des stocks d'équipements lors des locations et des retours, la gestion des places disponibles dans les sessions de cours, et le suivi en temps réel du statut de paiement des factures.

VI. DIFFICULTÉS RENCONTRÉES

Gestion de l'ordre de création des tables : La principale difficulté technique a été de respecter l'ordre de création des tables pour éviter les erreurs de clés étrangères. Certaines tables font référence à d'autres qui n'étaient pas encore créées, ce qui nous a obligés à revoir plusieurs fois l'ordre du script.

Le type ENUM et les valeurs NULL : L'utilisation du type ENUM pour les champs comme état dans la table équipement ou statut_paiement dans facture_client a posé des problèmes lors de l'insertion de données. Une valeur non présente dans l'ENUM déclenche immédiatement une erreur MySQL.

L'alignement sur le modèle conceptuel : Au début du projet, chacune avait une interprétation légèrement différente du modèle entité-association. Une session de travail commune a été nécessaire pour aligner nos visions avant de commencer l'implémentation.

La gestion des versions du dictionnaire : Le dictionnaire de données a évolué plusieurs fois au cours du projet, notamment pour corriger des types de données incorrects et pour ajouter des attributs manquants. Ces modifications en cours de route ont eu des répercussions en cascade sur les scripts SQL.

VII. IMPLICATION DE CHAQUE MEMBRE

Ce projet a été mené de façon collaborative et équitable entre nous trois. Chaque étape clé, notamment la conception du MCD, la rédaction du dictionnaire des données et les choix de modélisation, a été discutée et validée collectivement avant que chacune prenne en charge sa partie d'implémentation.

Ben Ziki Nahid

- Participation à la conception collective du MCD et du MLD
- Participation à la rédaction et correction du dictionnaire des données
- Implémentation du script de création des tables (1_BD_schema.sql) : création des tables, contraintes, clés primaires et étrangères
- Implémentation du script d'insertion des données de test (2_BD_data.sql) : données réalistes et diversifiées
- Génération du diagramme EER via MySQL Workbench
- Participation à la rédaction du rapport

Gobalasamy Ashani

- Participation à la conception collective du MCD et du MLD
- Participation à la rédaction et correction du dictionnaire des données
- Rédaction des requêtes SELECT avancées d'exploration de la base de données (jointures, GROUP BY, HAVING, sous-requêtes)
- Conception et implémentation des vues (minimum 2)
- Conception et implémentation des procédures stockées
- Rédaction principale du rapport

Bennacar Salma

- Participation à la conception collective du MCD et du MLD
- Participation à la rédaction et correction du dictionnaire des données
- Conception et implémentation des triggers (minimum 2)
- Conception et implémentation d'une procédure stockée
- Conception et implémentation des fonctions SQL
- Participation à la rédaction du rapport

VIII. CONCLUSION

Ce projet nous a permis de concevoir et d'implémenter de A à Z une base de données relationnelle pour une entreprise fictive de gestion de stations de ski. On est partis des attentes du TP pour identifier les entités, construire le MCD, passer au schéma logique en 3NF, puis implémenter le tout sous MySQL.

Au-delà de la création des tables, on a pu mettre en pratique les fonctionnalités avancées vues en cours : les vues pour simplifier les requêtes récurrentes, les triggers pour automatiser les mises à jour critiques comme la disponibilité du matériel ou le statut des factures, et les procédures stockées pour sécuriser les opérations métier importantes comme la réservation ou la restitution d'équipements. On a aussi exploré les fonctions pour aller un peu plus loin et tester ce qu'on pouvait faire sur cette base.

Un des points sur lesquels on a le plus réfléchi c'est la modélisation des hiérarchies d'héritage, notamment pour les fournisseurs et les employés, et les choix de conception comme stocker la date de naissance plutôt qu'une tranche d'âge, ou figer le prix au moment de la réservation dans `prix_applique`.

Si on devait aller plus loin, on pourrait enrichir le modèle avec une gestion des interventions de maintenance liées aux fournisseurs de matériel, ou encore une gestion plus fine des horaires des employés. Ces pistes ont été évoquées mais volontairement mises de côté pour garder un modèle cohérent et maîtrisable dans le cadre de ce TP.